Building Distributed Systems in Go

Failures · concurrency · architecture · testing

D7024E · Practical lesson



Distributed systems fail in more than one way

PROCESS

Crash · restart · pause
Supervision · durable state · idempotent recovery

NETWORK

Lost · late · duplicate · reordered
Deadlines · retries · IDs · deduplication

TIME

Clock skew · stale data · expired work
Monotonic time · leases · explicit freshness

LOAD

Bursts · full queues · exhaustion
Bounds · backpressure · load shedding

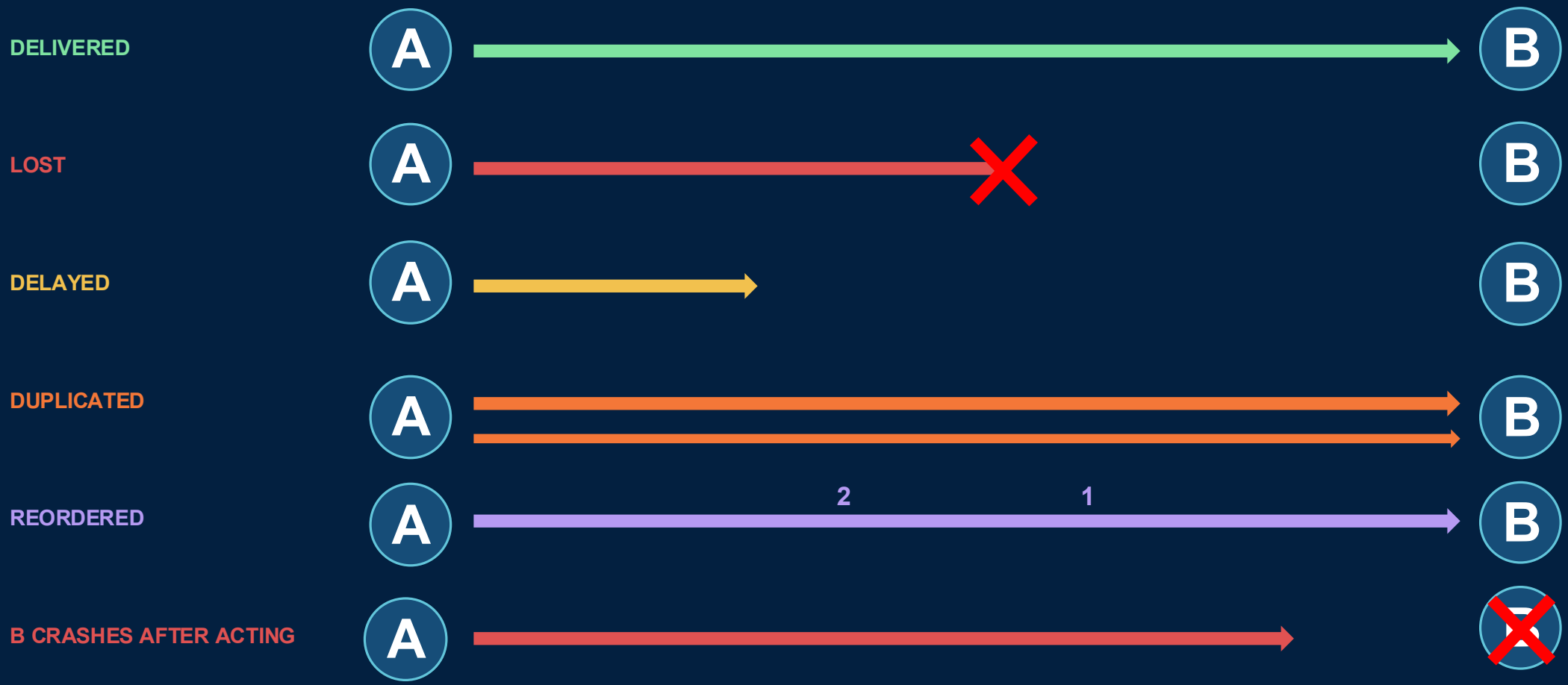
DATA

Partial writes · schema mismatch
Validation · transactions · versioned formats

CODE & OPERATIONS

Races · deadlocks · bad configuration
Ownership · CI · observability · rollback

One message, six possible stories



A resilience strategy is a trade-off

TIMEOUT

Limits how long we wait.
It does not stop work already running remotely.

RETRY

Can recover from loss.
Can also duplicate effects and amplify overload.

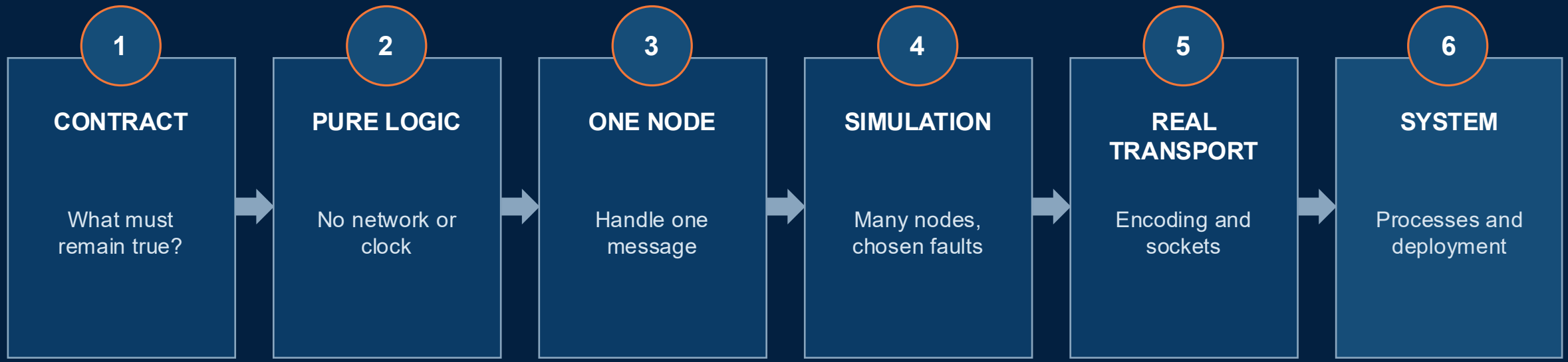
REPLICATION

Survives one machine failing.
Introduces consistency and coordination problems.

QUEUE

Absorbs a burst.
Adds delay and creates a backlog that must be bounded.

How to start: *shrink the world*



Go tools for concurrency

PART 1

Go in one slide

```go
package peer

type Message struct {
    ID   string
    From string
    Body []byte
}

type Sender interface {
    Send(Message) error
}

func (m Message) Validate() error {
    if m.ID == "" { return ErrMissingID }
    return nil
}
```

PACKAGE

A directory of related Go files.

STRUCT

Named data fields; values are copied unless a pointer is used.

METHOD

A function with a receiver: m.Validate().

INTERFACE

Satisfied implicitly by matching methods.

Errors are ordinary return values—handle them explicitly.

Start with behavior — *not a running process*

```go
func TestValidateRejectsMissingID(t *testing.T) {
    msg := Message{From: "peer-b"}

    err := msg.Validate()

    if !errors.Is(err, ErrMissingID) {
        t.Fatalf("got %v, want missing ID", err)
    }
}
```

ARRANGE

Create the smallest useful input.

ACT

Call one behavior directly.

ASSERT

Compare evidence with the contract.

A test is a small executable example. It does not need main(), ports, sockets, or another process.

Table-driven tests: many cases, one behavior

```go
func TestValidate(t *testing.T) {
    tests := []struct {
        name string
        msg  Message
        want error
    }{
        {"valid", Message{ID: "1", From: "b"}, nil},
        {"missing ID", Message{From: "b"}, ErrMissingID},
        {"missing sender", Message{ID: "1"}, ErrMissingSender},
    }

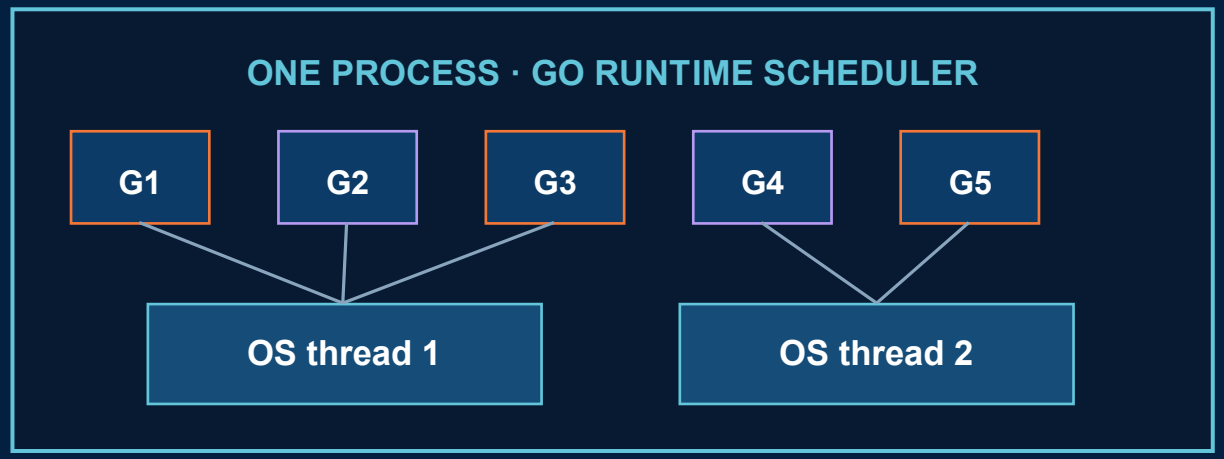
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := tt.msg.Validate(); !errors.Is(got, tt.want) {
                t.Fatalf("got %v, want %v", got, tt.want)
            }
        })
    }
}
```

WHY THIS STYLE?

Readable cases
Shared setup
Independent subtests
Easy failure additions

go test ./...

Goroutines are not threads



- An OS thread is an execution resource scheduled by the operating system
- A goroutine is a lightweight task scheduled by the Go runtime
- Many goroutines share a smaller set of OS threads — not one thread each
- Goroutines can still race when they access the same memory

Go routine example 1

```go
package main

func work(name string, delay time.Duration) {
    for i := 1; i <= 3; i++ {
        fmt.Printf("%s: step %d\n", name, i)
        time.Sleep(delay)
    }
}

func main() {
    go work("worker-a", 100*time.Millisecond)
    go work("worker-b", 150*time.Millisecond)

    // how long is long enough?
    time.Sleep(600 * time.Millisecond)
    fmt.Println("main: done")
}
```

GO STATEMENT

Starts the call in a new goroutine; main continues immediately.

OUTPUT

```
$ go run main.go
worker-b: step 1
worker-a: step 1
worker-a: step 2
worker-b: step 2
worker-a: step 3
worker-b: step 3
main: done
```

SCHEDULING

The runtime picks the interleaving; order can vary between runs.

DISCUSS · 60 SECONDS

Go routine example 2

```go
package main

func slowWorker() {
    fmt.Println("worker: starting")
    time.Sleep(200 * time.Millisecond)
    fmt.Println("worker: done")
}

func main() {
    go slowWorker()
    fmt.Println("main: exiting")
}
```

Go routine example 2

```go
package main

func slowWorker() {
    fmt.Println("worker: starting")
    time.Sleep(200 * time.Millisecond)
    fmt.Println("worker: done")
}

func main() {
    go slowWorker()
    fmt.Println("main: exiting")
}
```

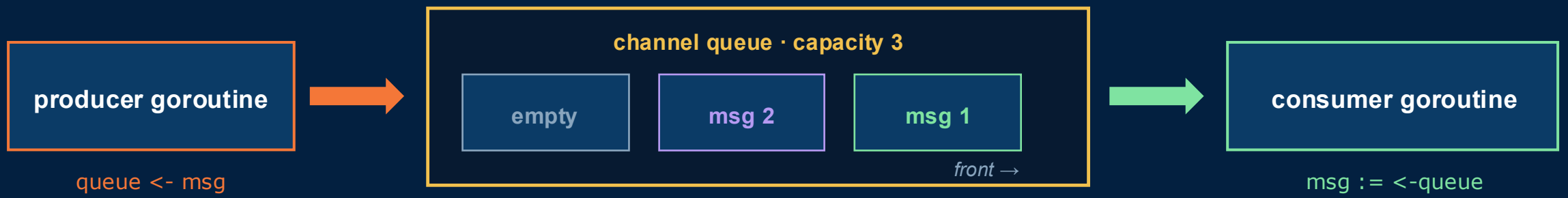
```
$ go run main.go
main: exiting

 ... worker lines never appear
```



Where did the worker's output go?
What decides when a Go program ends?

Channels

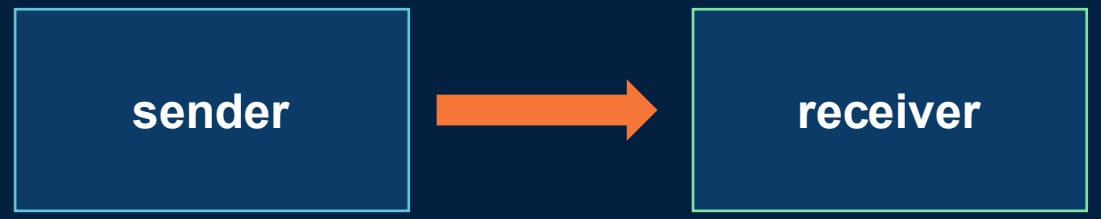


A buffered channel is a typed bounded FIFO queue plus synchronization: send waits when full; receive waits when empty.

Channel: transfer a value — *and synchronize*

```
messages := make(chan Message)

messages <- msg       // send
received := <-messages // receive
```



VALUE

A typed value crosses the boundary.

SYNCHRONIZATION

A send or receive may block until progress is possible.

A channel is both a communication mechanism and a scheduling decision.

A goroutine needs an owner, a stop signal, and a join

THE WORKER

```
func worker(
    in <-chan Message,
    done chan<- bool,
) {
    // work until the owner closes in
    for msg := range in {
        handle(msg)
    }

    // join: tell the owner we are done
    done <- true
}
```

THE OWNER

```
in := make(chan Message, 10)
done := make(chan bool)

go worker(in, done)   // START

for _, m := range msgs {
    in <- m
}

close(in)   // STOP: no more work
<-done      // JOIN: wait for the worker
```

The owner starts the goroutine, decides when it stops, and waits for it to finish.

Unbuffered, buffered, and closed channels

UNBUFFERED

make(chan Message)

A rendezvous: send waits for a receiver.

BUFFERED

make(chan Message, 10)

A bounded queue: send waits only when full.

CLOSED

close(ch)

No more values will be sent. Sending now panics.

- The producer that owns completion closes the channel
- A receiver normally does not close a channel
- Close is not cancellation; use context for cancellation
- Receive with value, ok := <-ch when closure matters

send after close

```
panic: send on closed channel
```

Buffer capacity and channel ownership are part of the contract

Challenge: this looks fast — *what can go wrong?*

```
type Server struct {
    peers     map[string]time.Time
    transport Transport
}

func (s *Server) Handle(m Message) {
    go func() {
        s.peers[m.From] = time.Now()
        _ = s.transport.Send(
            m.From,
            Ack{ID: m.ID},
        )
    }()
}
```

DISCUSS · 60 SECONDS

?

Find at least three failure modes

Assume Handle may be called concurrently

Challenge: this looks fast — *what can go wrong?*

```go
type Server struct {
    peers     map[string]time.Time
    transport Transport
}

func (s *Server) Handle(m Message) {
    go func() {                    // ①
        s.peers[m.From] = time.Now() // ② ③
        _ = s.transport.Send(      // ④ ⑤
            m.From,
            Ack{ID: m.ID},
        )
    }()
}
```

① **GOROUTINE · NO OWNER OR BOUND**

A burst creates goroutines the caller cannot stop or join.

② **MAP WRITE · DATA RACE**

Concurrent handlers mutate the same map without synchronization.

③ **time.Now · HIDDEN CLOCK**

The wall clock cannot be controlled by a deterministic unit test.

④ **SEND CALL · NO LIFETIME**

Send receives no context or deadline and may wait indefinitely.

⑤ **_ = · LOST EVIDENCE**

The underscore deliberately discards the network error.

context.Context: the lifetime of one operation

```
type Context interface {
    Deadline() (time.Time, bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

DEADLINE

When should this operation stop waiting?

CANCELLATION

A signal that work on this request should stop.

REQUEST VALUES

Trace or request IDs—not optional function parameters.

Context is an interface, not a struct. Pass it explicitly as the first parameter.

Three common context patterns

STOP A LOOP

```
for {
    select {
    case job, ok := <-jobs:
        if !ok { return nil }
        handle(job)
    case <-ctx.Done():
        return ctx.Err()
    }
}
```

The loop observes cancellation while waiting.

BOUND ONE CALL

```
opCtx, cancel := context.WithTimeout(
    ctx,
    250*time.Millisecond,
)
defer cancel()

return lookup(opCtx, key)
```

The child may finish sooner than its parent.

PASS IT ON

```
func Handle(ctx context.Context,
                msg Message) error {
    return node.Lookup(ctx, msg.Key)
}

func (n *Node) Lookup(ctx context.Context,
    key string) error {
    return n.sender.Send(ctx,
                         peer,
                         Request{Key: key})
}
```

Every blocking boundary receives the same lifetime.

Each WithCancel or WithTimeout call adds a child

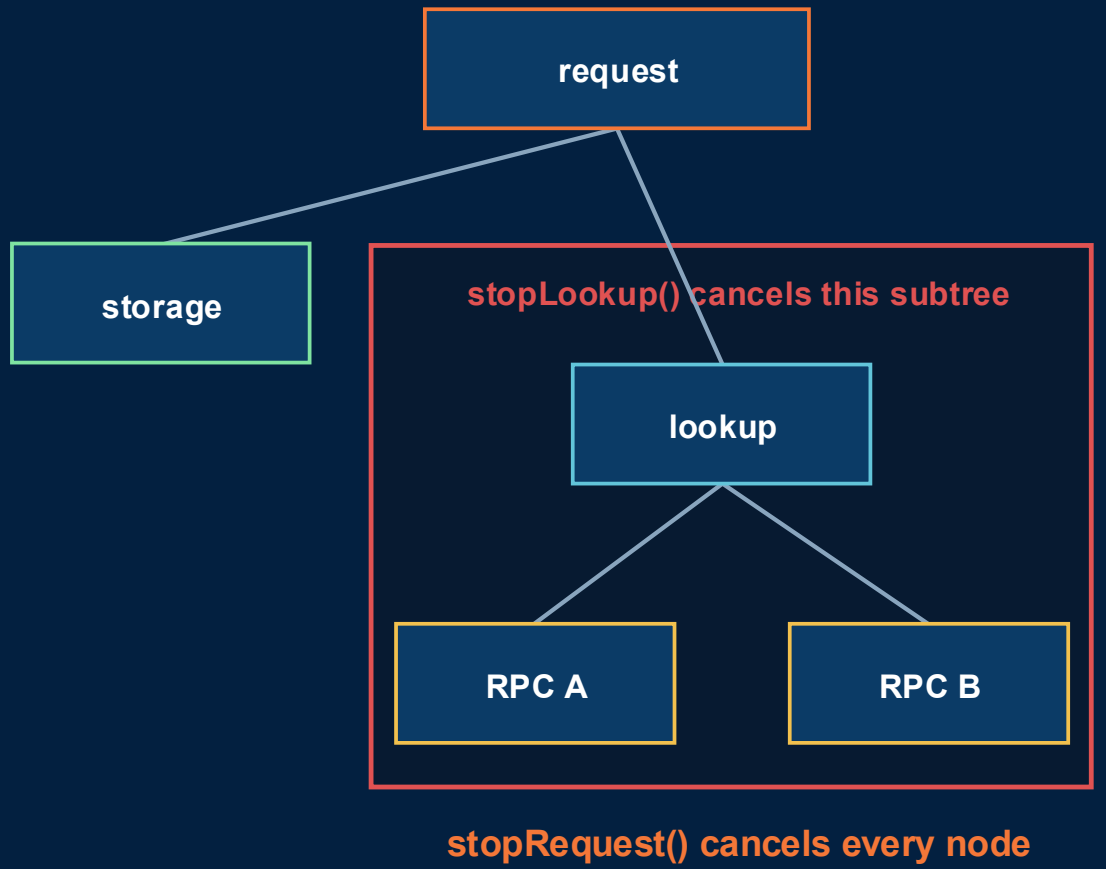
```
request, stopRequest := context.WithCancel(parent)
defer stopRequest()

storage, stopStorage := context.WithTimeout(
    request, 300*time.Millisecond,
)
defer stopStorage()

lookup, stopLookup := context.WithCancel(request)
defer stopLookup()

rpcA, stopA := context.WithTimeout(
    lookup, 200*time.Millisecond,
)
defer stopA()

rpcB, stopB := context.WithTimeout(
    lookup, 500*time.Millisecond,
)
defer stopB()
```



Use context at API and goroutine boundaries

```go
func Fetch(ctx context.Context, peer Peer) (Message, error) {
    ctx, cancel := context.WithTimeout(
        ctx, 200*time.Millisecond,
    )
    defer cancel()

    response := make(chan Message, 1)
    go request(ctx, peer, response)

    select {
    case msg := <-response:
        return msg, nil
    case <-ctx.Done():
        return Message{}, ctx.Err()
    }
}
```

- Pass ctx first; do not store it in a struct.
- Always call the returned cancel function.
- Never pass nil; use context.TODO while designing.
- Propagate ctx into outgoing calls.
- A deadline limits waiting; remote work may still finish.

A full channel forces an overload decision

WAIT OR CANCEL

```
select {
case inbox <- msg:
    return nil
case <-ctx.Done():
    return ctx.Err()
}
```

Preserves the message and propagates backpressure.

REJECT

```
select {
case inbox <- msg:
    return nil
default:
    return ErrOverloaded
}
```

Fails fast so the caller can retry or shed work.

DROP

```
select {
case metrics <- sample:
default:
    // intentionally lossy
}
```

Use only when the contract explicitly permits loss.

Buffer size and full-queue behavior belong in the API contract—not in a hidden tuning constant.

WaitGroup: wait until a group of goroutines finishes

```go
func processAll(ctx context.Context, messages []Message) {
    var wg sync.WaitGroup

    for _, msg := range messages {
        wg.Add(1) // increment before starting
        go func(msg Message) {
            defer wg.Done()
            handle(ctx, msg)
        }(msg)
    }

    wg.Wait()
}
```



ADD

Increment before the go statement.

DONE

Use defer so every return decrements.

WAIT

Blocks until the counter is zero.

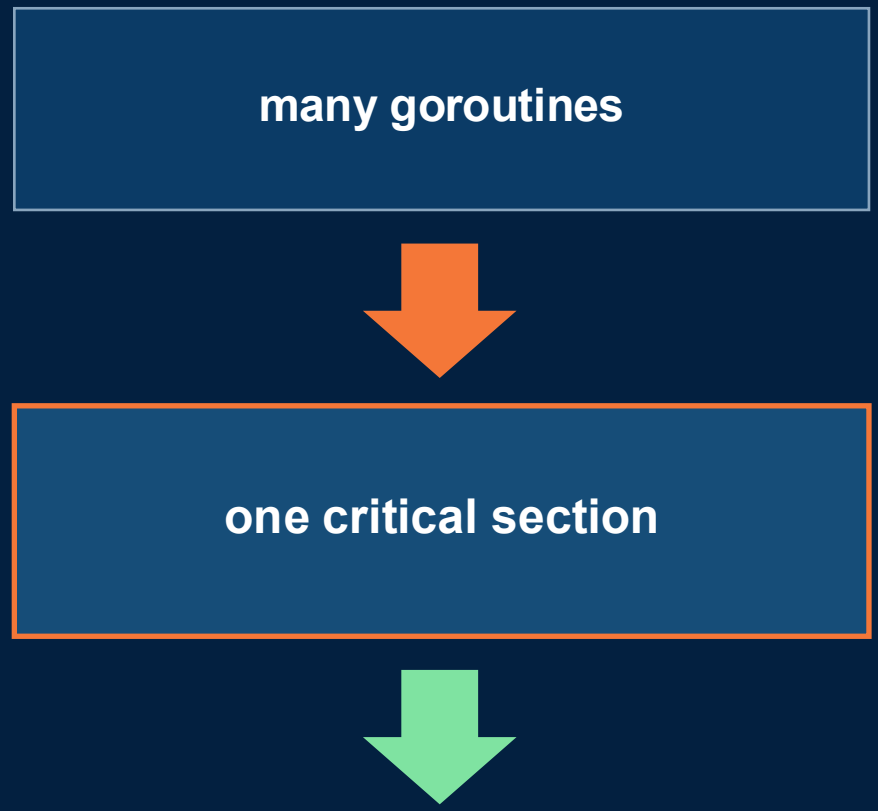
Context asks workers to stop; WaitGroup lets their owner observe that they have stopped.

Mutex: protect an invariant—not merely a line

```
type PeerSet struct {
    mu       sync.Mutex
    lastSeen map[string]time.Time
}

func (p *PeerSet) Touch(id string, now time.Time) {
    p.mu.Lock()
    defer p.mu.Unlock()

    p.lastSeen[id] = now
}
```



Invariant: the map is accessed only while mu is held.

Mutex rule: never wait on the network while locked

DANGEROUS

```
p.mu.Lock()
defer p.mu.Unlock()

p.lastSeen[id] = now
err := transport.Send(ctx, peer, msg)
// lock is held while the network waits
```

BOUNDED

```
p.mu.Lock()
p.lastSeen[id] = now
snapshot := p.copyNeededState()
p.mu.Unlock()

err := transport.Send(ctx, peer, makeMsg(snapshot))
```

A mutex should guard a short in-memory invariant. It must not become a distributed lock.

Channel or mutex? Choose by ownership

USE A MUTEX

Several goroutines share a small in-memory invariant.

Examples: cache, map, counter.

Do not force channels onto every problem.

USE A CHANNEL

Transfer work or ownership to another goroutine.

Examples: event loop, bounded worker queue.

Do not use a channel as a disguised global variable.

The key question is: who owns the mutable state, and when may ownership change?

Deadlock 1: a send with no receiver

```go
package main

func main() {
    ch := make(chan int)

    ch <- 1          // blocks: nobody receives
    fmt.Println(<-ch) // never reached
}
```

```
$ go run main.go
fatal error: all goroutines are
asleep - deadlock!

goroutine 1 [chan send]:
main.main()
```

THE RENDEZVOUS FAILS

An unbuffered send completes only when a receiver is ready. main is the only goroutine — the partner it waits for is the line below.

THE RUNTIME NOTICES

Go crashes only when every goroutine is asleep. If one unrelated goroutine still runs, this deadlock hides silently.

TWO FIXES

`go func() { ch <- 1 }()` or `make(chan int, 1)`
a partner goroutine — or one slot of buffer.

A deadlock is not an error in a line of code — it is a wait that no other goroutine will ever end.

Deadlock 2: two locks, opposite order

```
func transfer(from, to *Account, amount int) {
    from.mu.Lock()
    defer from.mu.Unlock()

    to.mu.Lock()   // waits for the other transfer
    defer to.mu.Unlock()

    from.balance -= amount
    to.balance += amount
}

// two transfers, opposite directions
go transfer(a, b, 10) // locks a … wants b
go transfer(b, a, 20) // locks b … wants a
```

```
fatal error: all goroutines are asleep - deadlock!
goroutine 17 [sync.Mutex.Lock]:
goroutine 18 [sync.Mutex.Lock]:
```

THE CYCLE

Goroutine 17 holds a and waits for b; goroutine 18 holds b and waits for a. A circle of waiting that no one can leave.

WHY IT HIDES

Both transfers are individually correct. The bug exists only in their combination — and only in one unlucky interleaving.

THE FIX: ONE ORDER

Lock accounts in a global order (lowest ID first) — or give all balances a single owner goroutine.

Every deadlock is a cycle. Break it with a global lock order — or remove it with a single owner.

Fix: lock in a global order — lowest ID first

```
func transfer(from, to *Account, amount int) {
    first, second := from, to
    if to.id < from.id { // lowest ID first
        first, second = to, from
    }

    first.mu.Lock()
    defer first.mu.Unlock()
    second.mu.Lock()
    defer second.mu.Unlock()

    from.balance -= amount
    to.balance += amount
}

go transfer(a, b, 10) // locks a, then b
go transfer(b, a, 20) // ALSO a, then b
```

```
$ go run -race main.go   a=110 b=90   — every run
```

SAME CALLS, ONE ORDER

Both transfers now lock a before b, whatever direction the money flows. One may wait — but a cycle can never form.

WHY IT WORKS

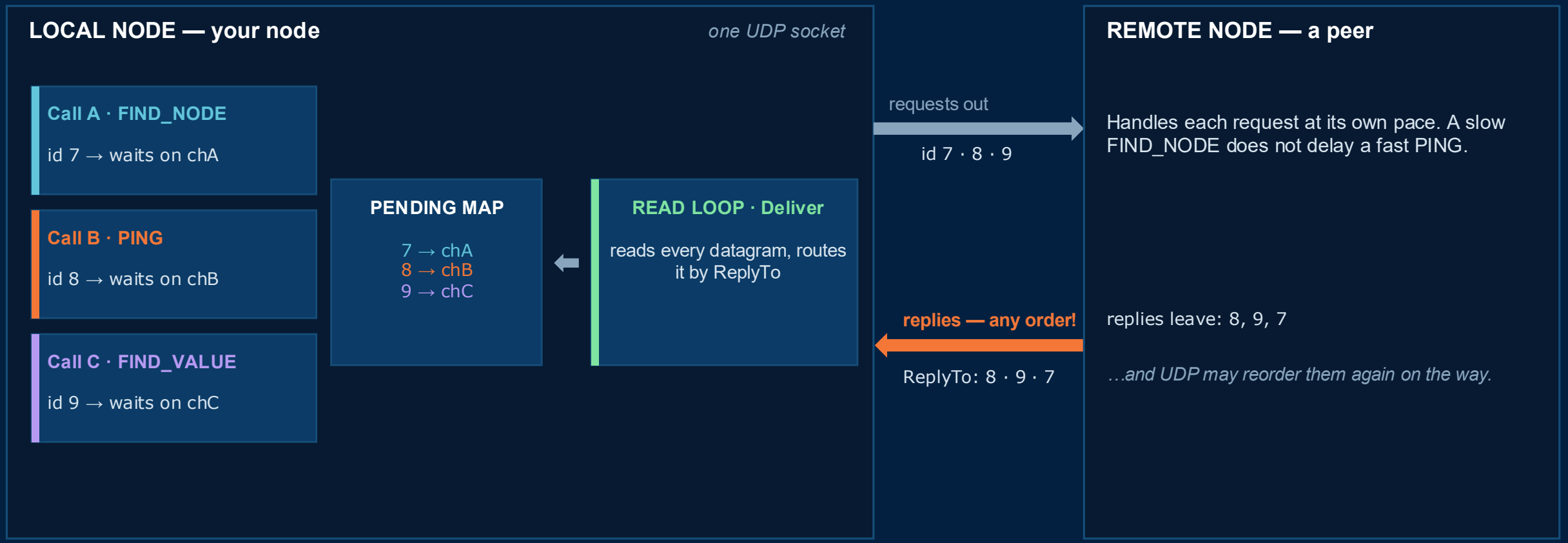
With a total order, every wait points upward — toward a higher ID. A cycle would need one edge pointing down, and there is none.

THE COST

The order lives in discipline, not in the type system — nothing stops new code from breaking it. One owner (slide 28) removes the rule entirely.

A total order kills cycles of every length — not just this pair, but any chain of accounts.

Node communication



Challenge: the pending map is locked — *is this safe?*

DISCUSS · 90 SECONDS

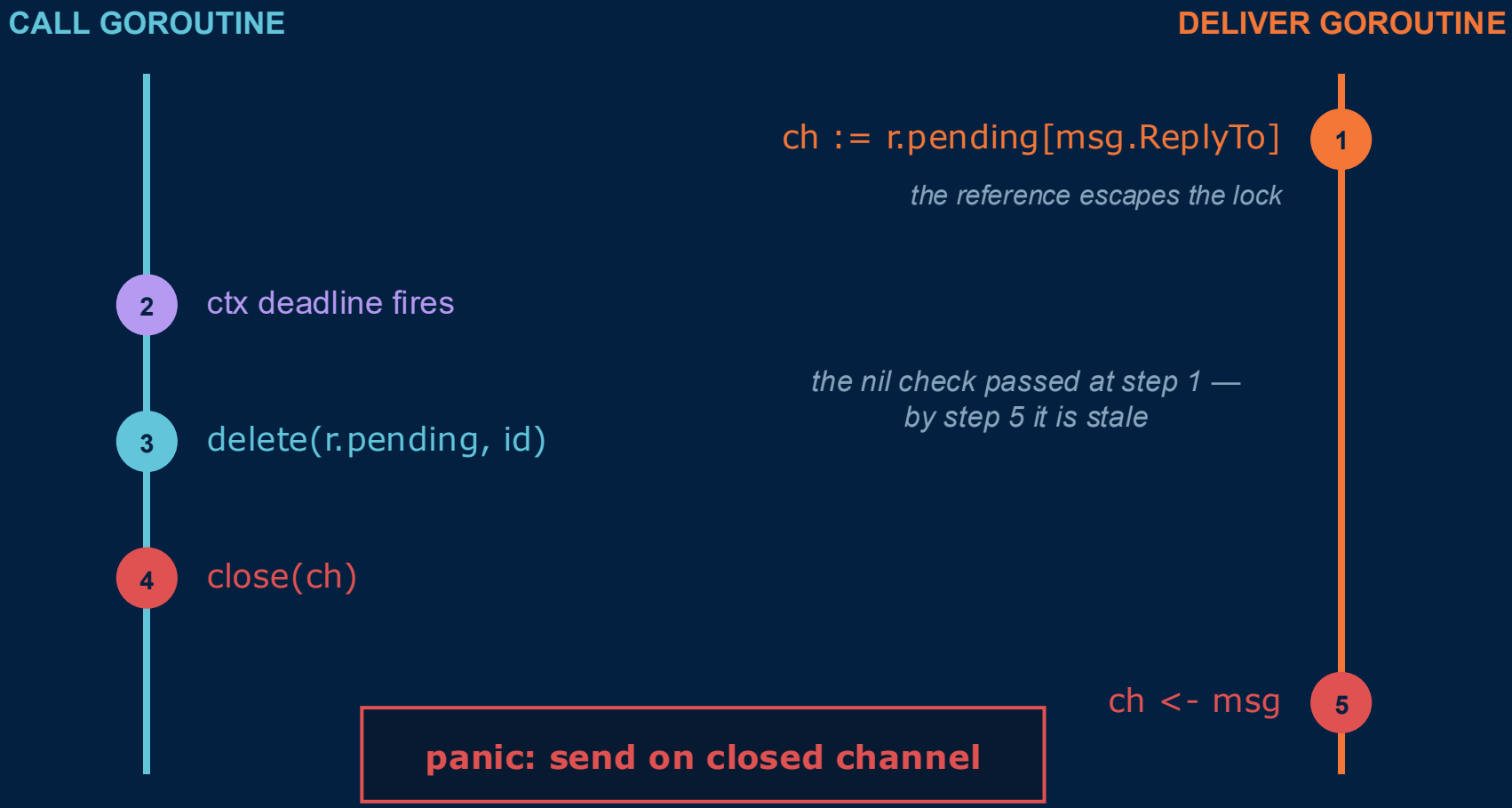
```
// called from any goroutine that wants a reply
func (r *RPC) Call(
    ctx context.Context, req Message,
) (Message, error) {
    ch := make(chan Message)
    r.mu.Lock()              // register
    r.nextID++
    id := r.nextID
    r.pending[id] = ch
    r.mu.Unlock()

    defer func() {        // deregister
        r.mu.Lock()
        delete(r.pending, id)
        r.mu.Unlock()
        close(ch)
    }()

    r.transport.Send(req.WithID(id))
    select {
    case reply := <-ch:
        return reply, nil
    case <-ctx.Done():
        return Message{}, ctx.Err()
    }
}
```

```
// called by the transport read loop
func (r *RPC) Deliver(msg Message) {
    r.mu.Lock()
    ch := r.pending[msg.ReplyTo]
    r.mu.Unlock()
    if ch != nil { ch <- msg }
}
```

Reveal: a legal interleaving still panics



Data-race-free does not mean concurrency-correct. Locks protect memory—not protocol lifetimes.

Fix: transfer pending-call ownership exactly once

```go
// called from any goroutine that wants a reply
func (r *RPC) Call(
    ctx context.Context, req Message,
) (Message, error) {
    ch := make(chan Message, 1)  // one reply can wait

    r.mu.Lock()              // register — unchanged
    r.nextID++
    id := r.nextID
    r.pending[id] = ch
    r.mu.Unlock()

    defer r.take(id)      // deregister — never close

    r.transport.Send(req.WithID(id))

    select {
    case reply := <-ch:
        return reply, nil
    case <-ctx.Done():
        return Message{}, ctx.Err()
    }
}
```

```go
// removes and returns the entry — one winner
func (r *RPC) take(id ID) (chan Message, bool) {
    r.mu.Lock()
    defer r.mu.Unlock()
    ch, ok := r.pending[id]
    if ok { delete(r.pending, id) }
    return ch, ok
}

// called by the transport read loop
func (r *RPC) Deliver(msg Message) {
    if ch, ok := r.take(msg.ReplyTo); ok {
        ch <- msg // cannot block
    }
}
```

RESPONSE WINS Deliver takes the entry, sends into the buffer.

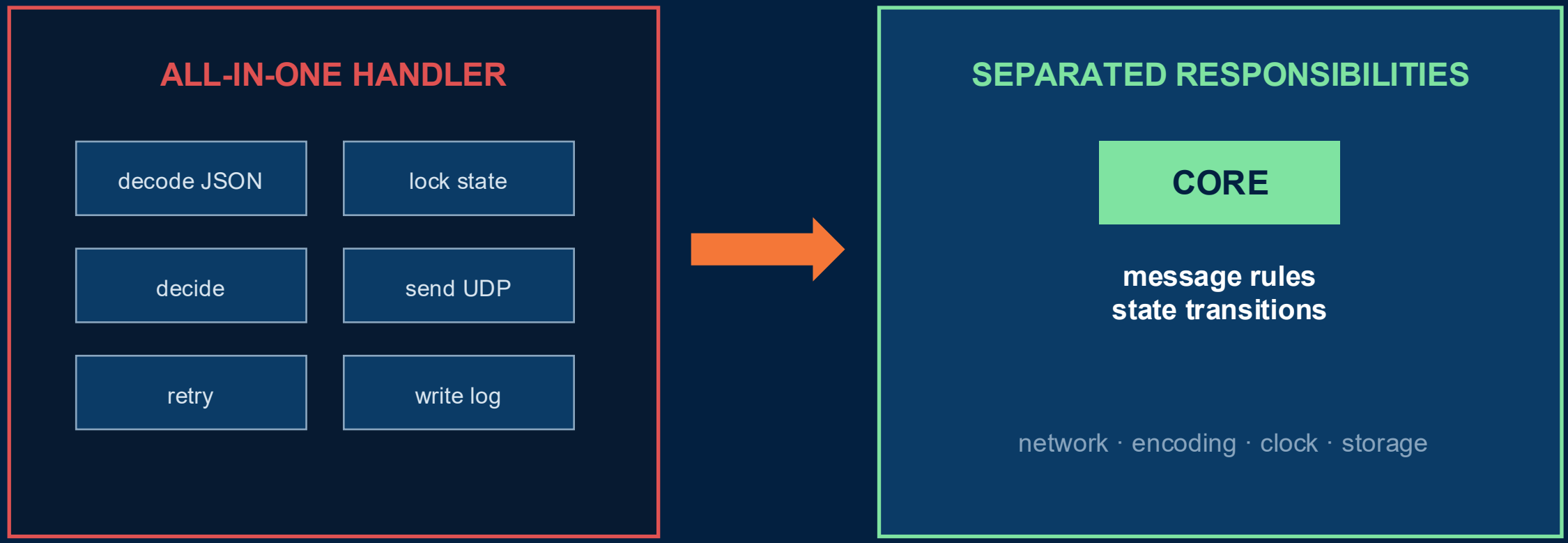
TIMEOUT WINS deferred take removes it; late reply dropped.

NO CLOSE nobody closes ch; the GC reclaims it after the call.

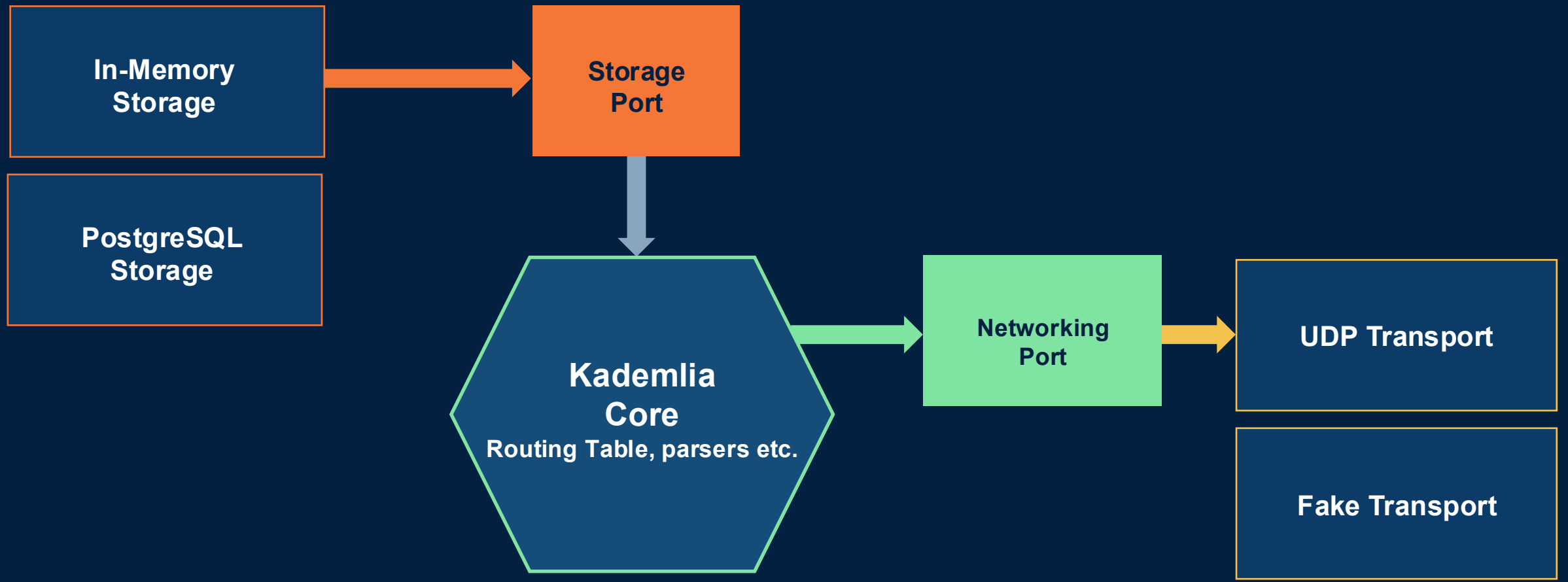
Architecture as failure isolation

PART 2

What should the core know about sockets?



Kademlia Hexagon architecture



The real adapter: a Network port, a socket struct

THE PORT

```go
// the port: what the core consumes
type Network interface {
    Send(to Addr, msg Message) error
    Close() error
}

// the adapter: a struct owning a socket
type UDPNetwork struct {
    conn *net.UDPConn // the socket
    node *Node        // inbound target
}
```

THE WIRING

```go
// wiring: open the socket, start the loop
func Listen(
    local string, node *Node,
) (*UDPNetwork, error) {

    addr, err := net.ResolveUDPAddr("udp", local)
    if err != nil { return nil, err }

    conn, err := net.ListenUDP("udp", addr)
    if err != nil { return nil, err }

    n := &UDPNetwork{conn: conn, node: node}
    go n.readLoop() // adapter owns the goroutine
    return n, nil
}
```

The mock from the seam slide satisfies the same Network interface — that is the whole trick.

The core sees Network. Only the adapter knows there is a socket, an OS, and a goroutine inside.

The adapter's two directions: Send out, Deliver in

OUTBOUND · Send

```go
// outbound: messages become bytes
func (n *UDPNetwork) Send(
    to Addr, msg Message,
) error {
    b, err := encode(msg)
    if err != nil { return err }

    _, err = n.conn.WriteToUDP(b, to.UDP())
    return err
}

func (n *UDPNetwork) Close() error {
    return n.conn.Close() // ends the loop
}
```

INBOUND · read loop

```go
// inbound: bytes become messages
func (n *UDPNetwork) readLoop() {
    buf := make([]byte, 65536)
    for {
        ln, _, err := n.conn.ReadFromUDP(buf)
        if err != nil {
            return // socket closed: loop ends
        }

        msg, err := decode(buf[:ln])
        if err != nil {
            continue // bad datagram: drop it
        }

        n.node.Deliver(msg)
    }
}
```

Same Message type on both sides — encode and decode are the only translation.

Send turns messages into bytes; the read loop turns bytes back into messages for Deliver.

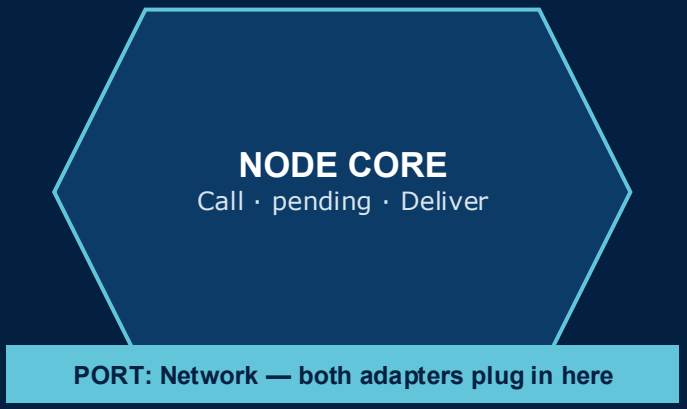
The channel adapter: same port, no sockets

CHANNEL ADAPTER

```
type ChanNetwork struct {
    local Addr
    wires map[Addr]chan Message
}

func NewChanNetwork(
    local Addr, wires map[Addr]chan Message, nd *Node,
) *ChanNetwork {
    c := &ChanNetwork{local, wires}
    go c.readLoop(nd) // the adapter owns it
    return c
}

func (c *ChanNetwork) Send(to Addr, msg Message) error {
    w, ok := c.wires[to]
    if !ok { return ErrUnknownPeer }
    msg.From = c.local // replies know the sender
    w <- msg
    return nil
}
```




```
func (c *ChanNetwork) readLoop(nd *Node) {
    for msg := range c.wires[c.local] {
        nd.Deliver(msg) // the same door as UDP
    }
}

func (c *ChanNetwork) Close() error {
    close(c.wires[c.local]) // range ends: loop exits
    return nil
}
```

Owner, stop signal, join: NewChanNetwork starts the loop, Close ends it — slide 16, kept honest.

Configure the adapter, then connect the core

1 · SELECT THE ADAPTER

```go
// choose the wire once, from configuration
func NewNetwork(
    cfg Config, node *Node,
) (Network, error) {

    switch cfg.Net {
    case "udp":
        return Listen(cfg.Addr, node)

    case "fake":
        return NewChanNetwork(
            cfg.Addr, cfg.Wires, node), nil
    }

    return nil, fmt.Errorf("bad net %q", cfg.Net)
}
```

2 · CONNECT THE CORE

```go
// main: configure, connect, run
node := NewNode(cfg.ID)

net, err := NewNetwork(cfg, node)
if err != nil { log.Fatal(err) }
defer net.Close()

node.Use(net) // Kademlia core gets its port

node.Bootstrap(cfg.Seed)
contacts, err := node.Lookup(ctx, target)
```

everything after Use is adapter-blind

go run . -net=udp -addr=:9001 on three machines — or -net=fake with three nodes in one test binary.

Configuration picks the adapter once, in main — the Kademlia logic never checks which one it got.

Kademlia lookup: core logic, any adapter

```go
// iterative FIND_NODE lookup (pseudo-Go)
func (n *Node) Lookup(
    ctx context.Context, target ID,
) ([]Contact, error) {
    short := n.routing.KClosest(target)

    for !short.Converged() {
        for _, c := range short.Next(alpha) {
            reply, err := n.Call(ctx, FindNode(c, target))
            if err != nil {
                short.MarkDead(c) // timeout, or ctx done
                continue
            }
            short.Add(reply.Contacts...)
        }
    }
    return short.KClosest(), ctx.Err()
}
```

SHORTLIST

The k best contacts seen so far, ordered by XOR distance to the target.

α PER ROUND

Each round queries α unvisited peers — goroutines, joined by a WaitGroup.

CONVERGED

A round that brings nothing closer ends the walk; return the k closest.

The lookup speaks only Call — whether replies travel over UDP or a channel is the adapter's business.

What the fake network buys you

SIMPLER

No sockets, ports, or privileges — the wires are a map of channels, built in three lines.

FASTER

No wall-clock waits: a whole cluster boots, runs a lookup, and shuts down in milliseconds.

FAULT INJECTION

Drop, delay, duplicate, reorder — every story from slide 3 becomes a test input, on demand.

PARTITIONS

Remove wires to split the network into islands; restore them and test how the protocol heals.

SCALABILITY

Hundreds of nodes in one process — no VMs, no port collisions, no cloud bill, no cleanup.

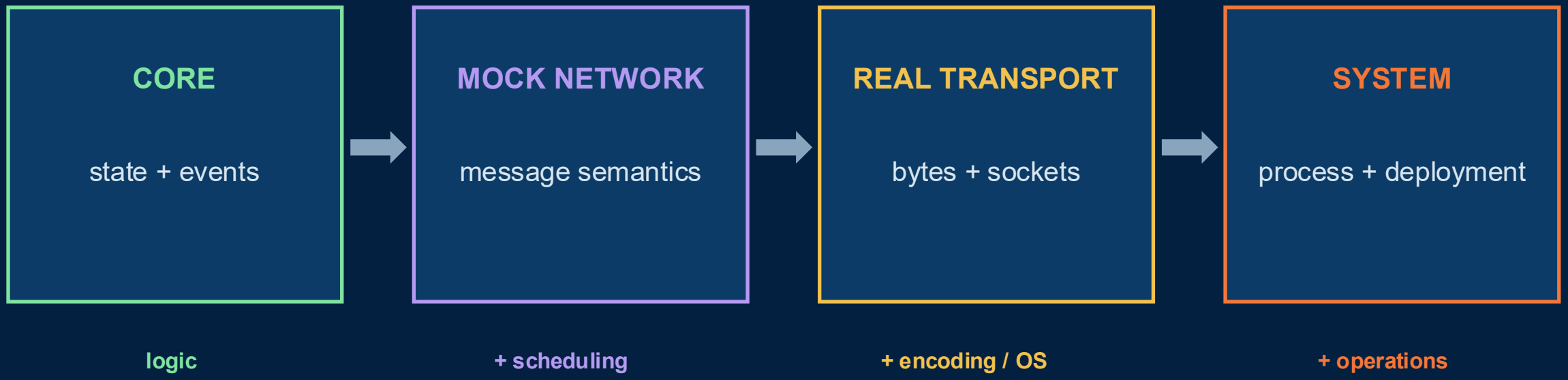
REPRODUCIBLE

A scripted scenario reruns identically in CI — network chaos becomes a table-driven test.

What it cannot catch: encoding bugs, MTU limits, kernel buffers, real latency — that is why exactly one adapter gets replaced next.

The fake network turns the network itself into test input.

Then replace exactly one adapter



When a later stage fails, the earlier evidence narrows where to investigate.

Before adding the real network, *ask…*

- ✓ Core logic runs without sockets
- ✓ Concurrency is bounded
- ✓ Context reaches outgoing work
- ✓ Tests are deterministic
- ✓ Late responses cannot block
- ✓ Messages are validated
- ✓ Every goroutine has an owner
- ✓ Duplicate effects are safe
- ✓ Adapters share a contract suite
- ✓ go test -race ./... is clean

Build the smallest world that can prove the next claim

Repository layout communicates boundaries

```
distsys/
├── go.mod
├── cmd/
│   └── node/
│       └── main.go          # composition root only
├── internal/
│   ├── core/                # state, rules, ports
│   ├── adapters/
│   │   ├── simnet/
│   │   └── udp/
│   └── storage/
│       └── memory/
├── api/                     # versioned wire formats
└── testdata/                # fixtures
```

https://github.com/eislab-cps/go-template

internal/

Compiler-enforced import boundary.

cmd/

Thin executable wiring—not reusable logic.

pkg/

Optional convention for intentionally supported external APIs.

Avoid catch-all packages named utils, common, or models.

internal is enforced; *pkg* is a promise

internal/

Code under x/internal/y may be imported only from within the tree rooted at x.

Go enforces this rule.

pkg/

The directory name has no special meaning to the Go compiler.

Use it only when external consumers should rely on the API.

Most application code should begin in internal/. A public package creates a compatibility obligation.

When does a second repository help?

SPLIT WHEN

Independent release cycle
Independent deployment
Different ownership or access
Public, versioned contract
Smaller security boundary

THE COST

Cross-repo changes are not atomic
Version coordination
Contract drift
Harder local development
More CI and release work

Start with one repository. Split at a real lifecycle boundary—not at a box in a diagram.